
Distributed Systems

Monsoon 2025

Lecture 12

International Institute of Information Technology

Hyderabad, India

Distributed Consensus

- Consider the following setting.
- There is one open position for **a top coder** for your start-up.
- **Four of you interview** a candidate independently.
- The four of you have a local decision on whether to hire this candidate or not.
- The four of you have to discuss and **arrive at a common decision**.
- This is the classical distributed consensus problem.

Distributed Consensus

- The problem of consensus is much more fundamental to distributed computing.
- Applications to
 - Distributed database transactions: **Commit/Abort**
 - Agreement on the **reading reported by different sensors**
 - **Air traffic control system**: all aircrafts must have the same view

Distributed Consensus

- Consider a distributed systems with n processes.
- Let us assume that there is logical connectivity between each pair of processes.
- These logical channels are assumed to be reliable.
- We will only consider the setting where **messages are not authenticated**.
 - So, message contents may be garbled, forged, etc.
 - Only the id of the sender is unforged through the network.
- The agreement variable is taken to be a Boolean value (0/1).

Distributed Consensus

- Two other significant assumptions on the system model:
- **Asynchronous/Synchronous**: We will consider both possibilities separately.
 - In the asynchronous case, if a message from a process P_i to P_j fails, then P_j cannot know the difference between the non-arrival of the message vs. a delayed arrival.
- Failure of processes: We consider that some of the processes may fail and in different ways:
 - **Fail-stop**
 - **Byzantine**
 - ...

Distributed Consensus

- Reconsider the hiring problem again.
- If one of the four who interview the candidate have a reason to influence the decision, we model that as a faulty process.
 - This is sometimes referred to as conflict-of-interest.

Formal Definition

- The **consensus** problem is defined as follows.
- Consider a distributed system with n processes.
- Each process has an initial value. All the correct processes agree on a single value subject to:
- **Agreement**: All **non-faulty** processes must agree on a single value
- **Validity**: If all the non-faulty processes have the same initial value, then the eventual agreed value by all the **non-faulty** processes must be that same value.
- **Termination**: Each **non-faulty** process must eventually decide on a value.

Distributed Consensus

- Some observations
- The **validity** condition **rules out trivial solutions** such as agreeing on a default value in all cases.
- The faulty processes may or may not decide – these are not considered in the solution.

A Variant – Distributed Agreement

- The formal problem definition is as follows:
- In a distributed system with n processes, we have one process designated as the **source** process that has an initial value.
- The source process and other processes have to reach an agreement about the value subject to:
 - **Agreement**: All **non-faulty** processes must agree on the same value
 - **Validity**: If the source process is non-faulty, then the eventual agreed value by all the **non-faulty** processes must be the initial value of the source process.
 - **Termination**: Each **non-faulty** process must eventually decide on a value.

Quick Recap

- We identified two important problems.
- Distributed Consensus and Distributed Agreement
- Both reducible to each other.
- Main difference:
 - Consensus: Every node has an initial value.
 - Agreement: Only one node, the initiator, has an initial value.
- Properties of Solution: Agreement, Validity, Termination.

Consensus -- One Table to Fill

Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous			
Asynchronous			

The Simple Case – Failure Free System

- Consider a setting where there are no faulty processors.
- In a fault-free setting, each process can broadcast its value to other processes.
 - An All-to-All broadcast.
- The decision can be reached by having all the processes compute a common function such as min/max/average/majority on the n values.

The Simple Case – Failure Free System

- Consider a setting where there are **no faulty processors**.
- In a fault-free setting, each process can broadcast its value to other processes.
 - An **All-to-All** broadcast
- The decision can be reached by having all the processes compute a **common function** such as min/max/average/majority on the n values.
- In the case of a **synchronous** network, all this can be achieved in a constant number of rounds.
- In the case of an **asynchronous** network, there is a possibility. Defer details to later.

One Table to Fill

Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous	Easy, All-to-All communication		
Asynchronous	Possible, to do later		

The Simple Case

- Given the simple nature of the problem in fault-free systems, we will now move to **faulty settings**.

Consensus : The Case of Crash Failures

- Let us consider the next difficult case – that of crash failures.
- In the crash failure, or the fail-stop model, a process may crash at any time during execution.
 - Even during the middle of executing a step including send/receive.
- Let us consider a system of n processes with up to f of them being faulty.
- We know the value of f , but not the exact number of faults.
- The value at each process is taken to be an integer.

Consensus in Fail-Stop Setting

- Process P_i has an initial value x_i .
- In each round, if P_i 's value changed in the previous round, P_i sends its value x_i to all other processes.
- P_i takes the **minimum** of all the received values and updates x_i to this value.
- If there are **f faulty** processes in the system, then the algorithm runs for **$f+1$** rounds.

Consensus in Fail-Stop Setting

The algorithm for Process P_i

Algorithm ConsensusFailStop

Begin

for round = 1 to $f+1$ do

begin

if the current value of x_i has not been sent yet

 broadcast(x)

y_j = the value received from P_j

$x_i = \min \{ x_i, \min_j \{ y_j \} \}$

end

output x_i as the consensus value

End

Example Run

- Consider a system of five nodes with initial values (1, 1, 0, 1, 1).
- The **consensus value**, if there are **no failures**, should be **0**.
- Even if the value of 0 is sent to one node before P3 fails, then 0 is still the consensus value and is valid.
- Let us assume that there is one faulty node, say P3.
- Suppose that in round 1, P3 fails!
- Suppose P3 sends its value to any one of the other nodes before failing, say P2.

Example Run

- Suppose P3 sends its value to any one of the other nodes before failing, say P2.
- Now, P2 takes the minimum of 1 and 0, and resets its value to 0.
- All others do not know of the value of P3 and update their values to 1.
- In the second round, P3 no longer sends any messages.
- But P2 has a value that it has not sent yet!
- So, P2 sends 0 to every one.
- All nodes update their value to 0.

Example Run

- Suppose P3 crashes before sending its value to any other node.
- It is similar to a system with just 4 nodes!
- Then, the only value that others learn is 1. So, the result produced is 1.

Consensus in Fail-Stop Setting

- We will show that the algorithm satisfies all the three conditions required.
- **Agreement**: Need all non-faulty processors to agree on the same value.
- In each faulty round, at least one process may become faulty.
- So, in $f+1$ rounds, there is **at least one round r** that is fault-free, i.e., has **no process failures**.
- In that round r , all non-faulty processes broadcast their current value.
- This value is used to set the minimum by all other non-faulty processes.

Consensus in Fail-Stop Setting

- **Agreement:** Need all non-faulty processors to agree on the same value.
- In each round, at least one process may become faulty.
- So, in $f+1$ rounds, there is at least one round r that has no process failures.
- In round r , all non-faulty processes broadcast their current value.
- This value is used to set the minimum by all other processes. Say the minimum value is v .
- Post this, only v may be broadcast by processors at most one more time.

Consensus in Fail-Stop Setting

- **Validity**: Need all non-faulty processors to agree on the same initial value if all such processors indeed start with an identical value.
- Since faults are fail-stop, processors that fail in a round r do not propagate incorrect values.
- This also means that a process that crashes has only sent correct values till the round it crashed.
- If all processors start with an identical value v , then v is the only value that is ever sent by any process.

Consensus in Fail-Stop Setting

- **Termination**: Need all non-faulty processors to eventually decide.
- Since the algorithm runs only for $f+1$ rounds, this condition too is met.

Consensus in Fail-Stop Setting

- Complexity:
- Number of rounds = $f+1$.
- Number of message per round = $O(n^2)$.
- Number of messages = $O(f.n^2)$
- This bound is tight. There are examples where this bound is met.
- Consider round r , for r between 1 and f , having one process fail after sending a message to just one other process.

Consensus in Fail-Stop Setting

- Lower Bound: If there are f faults, indeed $f+1$ rounds are required in the worst-case scenario.
- Consider the situation where every round exactly one process fails.

One Table to Fill

Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous	Easy, All-to-All communication	$f+1$ rounds	
Asynchronous	Possible, to do later		

The Byzantine Setting

- Let us now consider Byzantine faults and a synchronous setting.
- The name comes from the old Byzantine empire in history.
- Imagine that an army of invaders arranged in groups, each managed by a general, plans to attack from multiple directions.
- The attack is successful only if all the groups attack simultaneously.
- Requires the generals to reach an **agreement** on the time of attack.
- A general who is a traitor is modelled as a fault!

The Byzantine Setting

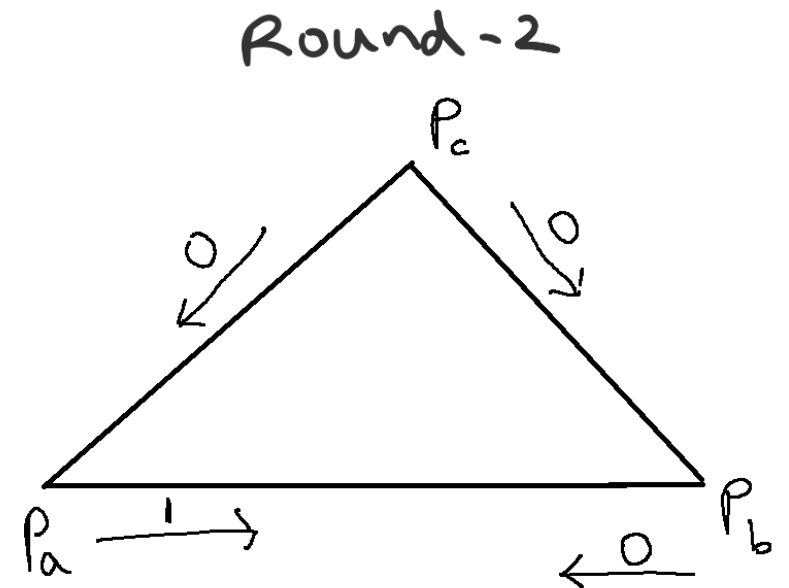
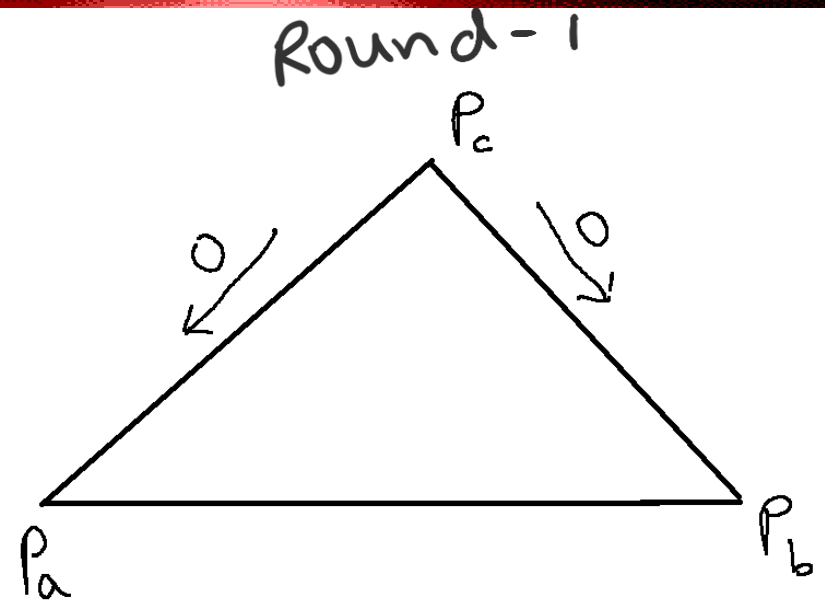
- The only way to communicate across the generals is by sending messages (via messengers)
- A faulty process, Byzantine general, may mislead one or more groups by sending different attacks times to different groups.
- Or, relay incorrect information.
- The problem of reaching an agreement in such settings is the agreement problem.

The Byzantine Setting

- Let us start a small example with 3 nodes and one of them being Byzantine faulty.
- Let us assume a synchronous fully-connected network topology.
- We use n to denote the number of nodes and f to denote the number of faulty nodes.
- In the example, $n = 3$ and $f = 1$.

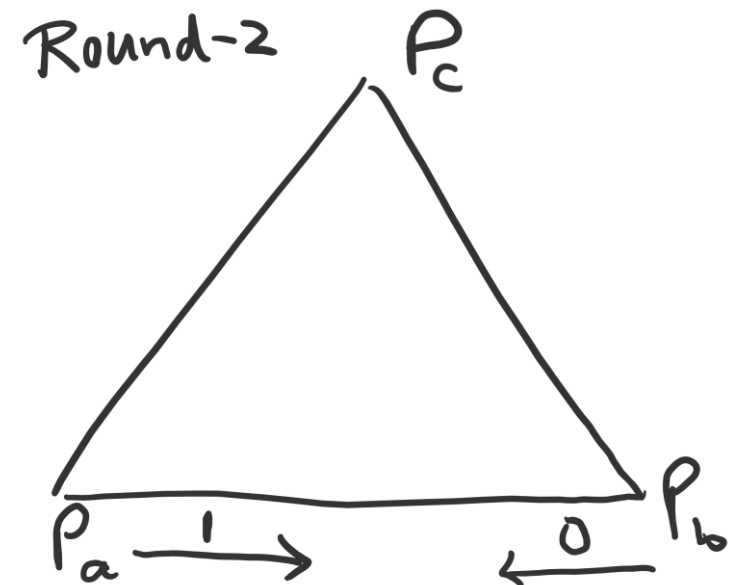
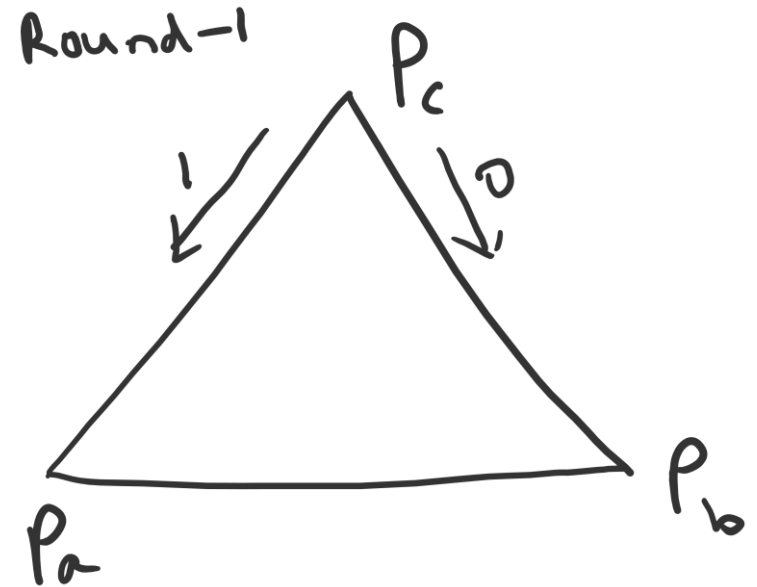
The Byzantine Setting

- Suppose that P_c is the initiator and P_a is the faulty node.
- The value at P_c is 0.
- In Round 1, P_c sends a value 0 to both P_a and P_b .
- In the Round 2, suppose that P_a sends 1 to P_b whereas P_b sends 0 to P_a .
- P_b has no clue as to which of P_a and P_c are at fault.
- Importantly, the same confusion exists if we assume that P_c and not P_a is faulty.



The Byzantine Setting

- Importantly, the same confusion exists if we assume that P_c and not P_a is faulty.
- Suppose that P_c is the initiator and P_c is the faulty node.
- The value at P_c is 0.
- In Round 1, P_c sends a value **1 to P_a** and a value **0 to P_b** .
- In the second round, **P_a relays 1 to P_b** whereas P_b sends 0 to P_a .
- P_b has no clue as to which of P_a and P_c are at fault.



The Byzantine Setting

- The trouble is that P_b gets identical inputs from P_c and P_a in two different scenarios with different output (validity) requirements.
- So, we conclude that one faulty node in a system of three nodes, agreement is not possible even in the synchronous setting.
- This result extends to also an n node system with $f = n/3$ or more.
- We establish a reduction as follows.

The Byzantine Setting

- Let $S(n, f)$ be a synchronous system of n nodes with f of them being faulty.
 - Our earlier example is the $S(3,1)$ system.
- Consider that f is at least $n/3$.
- Arrange the n nodes in $S(n, f)$ into three subsets S_1 , S_2 , and S_3 each of size $n/3$.
 - Assume that n divides 3.
- We now map $S(3,1)$ to these sets such that each node P_i in $S(3,1)$ simulates the set S_i , for $i = 1,2,3$.

The Byzantine Setting

- Arrange the n nodes in $S(n, f)$ into three subsets S_1 , S_2 , and S_3 each of size $n/3$.
 - Assume that n divides 3.
- We now map $S(3,1)$ to these sets such that each node P_i in $S(3,1)$ simulates the set S_i .
- If **$S(3, 1)$ has one faulty node**, then $S(n, f)$ has $n/3$ faulty nodes.
- The correct nodes in $S(3,1)$ simulate the actions of the other sets.
- Now, if there is an algorithm to reach agreement in $S(n, f)$ with $f \geq n/3$, then there is an algorithm for agreement in $S(3,1)$.
- Since the latter is impossible, so is the former.

Distributed Consensus

- The problem of consensus is much more fundamental to distributed computing.
- Applications to
 - Distributed database transactions: **Commit/Abort**
 - Agreement on the **reading reported by different sensors**
 - **Air traffic control system**: all aircrafts must have the same view

Distributed Systems

Monsoon 2025

Lecture 13

International Institute of Information Technology

Hyderabad, India

One Table to Fill

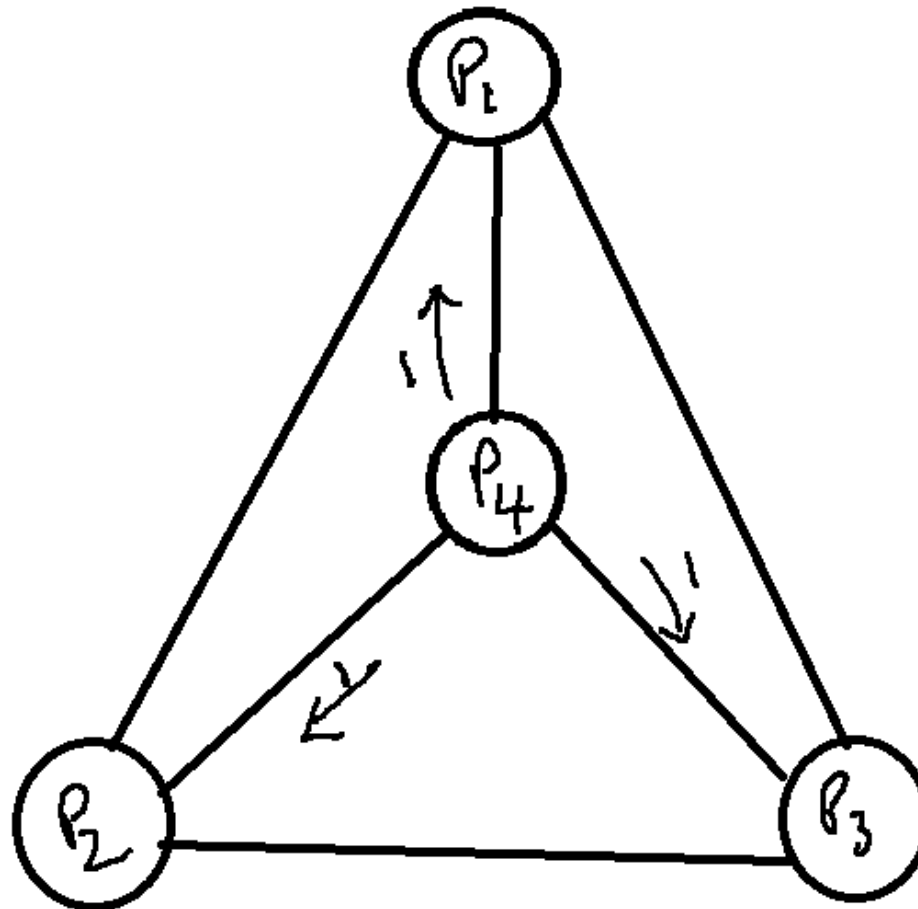
Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous	Easy, All-to-All communication	$f+1$ rounds	
Asynchronous	Possible, to do later		

An Algorithm

- Let us first see how agreement is possible in the $S(4,1)$ system.
- Notice that $S(4,1)$ has strictly less than $n/3$ faults.
- Consider that the initiator sends his value to all the other nodes.
 - A total of $n-1$ messages in this round
- In the second round, every other node sends the value it received from the initiator to all other nodes.
 - A total of $(n-1)(n-2)$ messages.
- At the end of the second round, each process takes the majority of all the inputs it received.
 - This majority serves to understand correctly the value at the initiator.

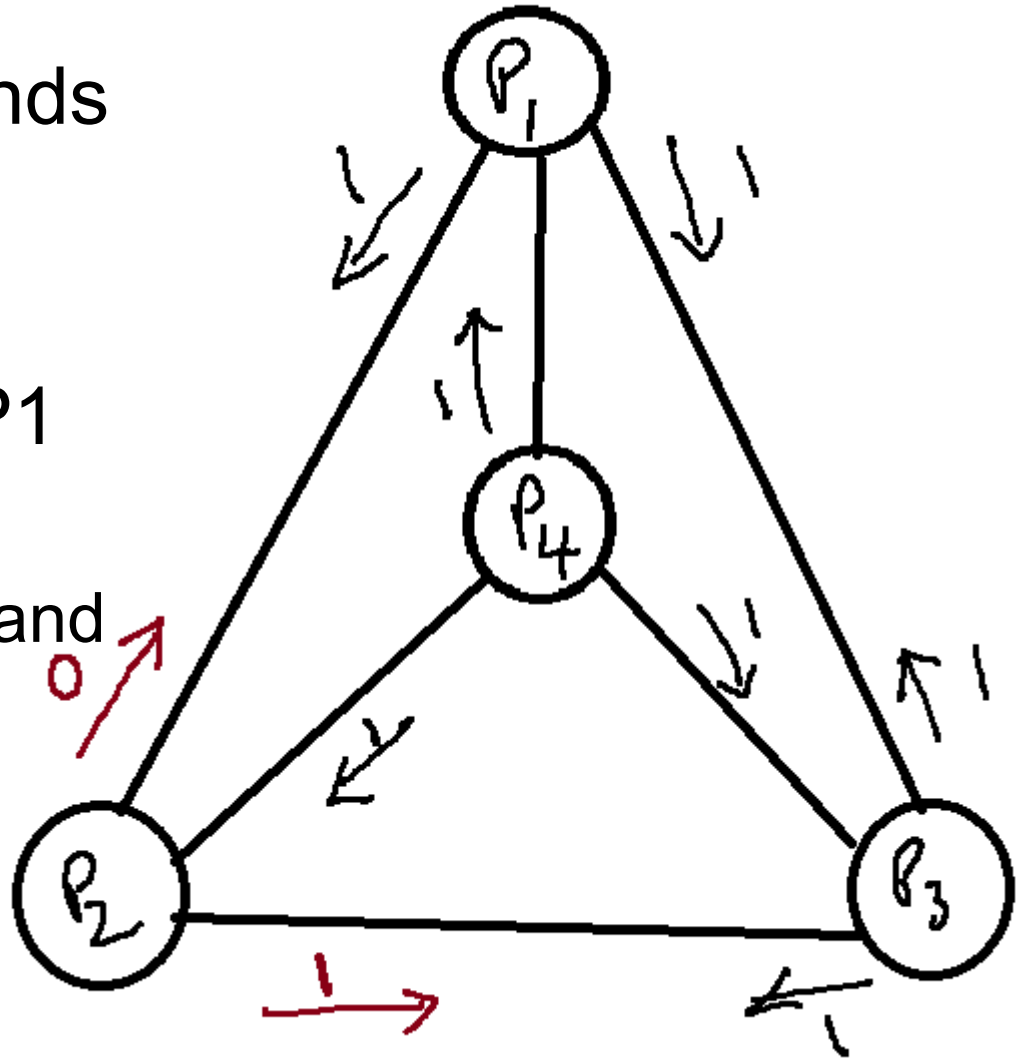
An Algorithm

- $S(4,1)$ in pictures.



$S(4, 1)$

- Round 2
- P2 is faulty, hence sends 1 to P3 and 0 to P1.
- Note the values that P1 receives.
 - 0 from P2, 1 from P3, and 1 from P4.



An Algorithm

- How is the algorithm working?
- Why did we need two rounds?
- The first round is clear – the initiator sends its value to everyone.
- Notice that there is at most one fault in the $S(4,1)$ system.
- So, by having everyone relay the message that they hear from the initiator and then taking a majority, the effect of the faulty node is washed out.
- In other words, two rounds required to tolerate one fault.
- Can think of the actions of each node in the second round as a distinct subproblem of $S(n-1, f-1)$.

An Algorithm

	P_1	P_2	P_3	P_4^*	P_5	P_6
$\overrightarrow{P_0^*}$	0	0	0	0	1	1

- Consider a seven node system with two faulty nodes.
- P_0 is the initiator and has initial value 0. Let us assume that P_0 is faulty.
- P_0 sends 0 to nodes P_1 through P_4 and 1 to others.

An Algorithm

$P_0 \downarrow$ $P_1 \rightarrow$	P_2	P_3	P_4	P_5	P_6
	0	0	0	0	0

$P_0 \downarrow$ $P_2 \rightarrow$	P_0	P_3	P_4	P_5	P_6
	0	0	0	0	0

$P_0 \downarrow$ $P_3 \rightarrow$	P_1	P_2	P_4	P_5	P_6
	0	0	0	0	0

$P_0 \downarrow$ $P_4 \rightarrow$	P_1	P_2	P_3	P_5	P_6
	0	0	1	0	0

$P_0 \downarrow$ $P_5 \rightarrow$	P_1	P_2	P_3	P_4	P_6
	1	1	1	1	1

$P_0 \downarrow$ $P_6 \rightarrow$	P_1	P_2	P_3	P_4	P_5
	1	1	1	1	1

- Consider the view of P_3 , say. It receives 0 from P_0 .
- In round 2, every one from P_1 to P_6 send the value they received from P_0 to everyone else.
 - But some could be faulty.
- Suppose that P_4 is faulty.
- In particular, P_4 sends 1 to P_3 and sends 0 to others.
- P_3 itself sends 5 messages with value 0 to everyone except P_0 and itself.

An Algorithm

	P_0	P_1	P_2	P_4	P_5	P_6		P_0	P_1	P_3	P_4	P_5	P_6
@ P_3	0	0	0	1	1	1	@ P_2	0	0	0	0	1	1

- At this stage, just taking majority might not help!
- Why?
- It is not clear who is at fault.
- P_3 cannot conclude that 0 is the “true value” and others sending 1 are faulty.
- Especially when the source is under fault.
- In particular, the output of different nodes at the nodes of this round can be different.
 - Not a valid consensus.

An Algorithm

- Let us see what the third round looks like.
- Each node (except P_0) send $(n-2)(n-3)$ messages conveying what they hear from each other in the previous round.

An Algorithm

$P_4 \downarrow$ P_2 P_3 P_5 P_6
 $P_1 \rightarrow$ 0 0 0 0

$P_4 \downarrow$ P_1 P_3 P_5 P_6
 $P_2 \rightarrow$ 0 0 0 0

$P_4 \downarrow$ P_1 P_2 P_5 P_6
 $P_3 \rightarrow$ 1 1 1 1

$P_4 \downarrow$ P_1 P_2 P_3 P_6
 $P_5 \rightarrow$ 0 0 0 0

$P_4 \downarrow$ P_1 P_2 P_3 P_5
 $P_6 \rightarrow$ 0 0 0 0

~~for~~ P_4 P_0 P_1 P_2 P_4 P_5 P_6
 At P_3 0 0 0 1 0 0

An Algorithm

- What if there are more nodes?
- What if there are more faulty nodes?
- How should the earlier algorithm be extended?
- Intuitively, need **f+1 rounds to wash out** the effect of all the (up to) f faulty nodes.
- In each round, each node acts as the initiator of a (distinct) $S(n_i, f_i)$ system with $n_i = n - i$ and $f_i = f - i$.

An Algorithm

- Intuitively, need $f+1$ rounds to wash out the effect of all the (up to) $f+1$ faulty nodes.
- In each round, each node acts as the initiator of a (distinct) $S(n_i, f_i)$ system with $n_i = n - i$ and $f_i = f - i$.
- Total number of **messages** across **rounds** i captured in the recurrence $M(n, f) = (n - 1) M(n-1, f-1)$, and $M(n, 1) = n-1$.
- Evaluates to $M(n, f) = (n - 1)(n - 2)(n - 3) \dots (n - f - 1) = O(n^f)$.

An Algorithm

- In particular, if the initiator is non-faulty, then that node sends the same value to all others.
- So, all non-faulty ones will all have the same value.
- Even through multiple rounds, the non-faulty processors continue to have the same notion of majority.

An Algorithm

- The initiator could be malicious and sends conflicting values to the non-faulty processes.
- The remaining system has $f - 1$ malicious processes, but all the loyal processes do not have the same view to begin with.
- Need the extra rounds to allow the non-faulty ones to produce a valid output.

An Algorithm

- Can use the above observations to show that even if the status of the initiator is likely to be malicious, the algorithm can tolerate up to f faults and arrive at an agreement if $n > 3.f$

One Table to Fill

Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous	Easy, All-to-All communication	$f+1$ rounds	Possible if $n > 3f$ $O(n^f)$ messages.
Asynchronous	Possible, to do later		

A Polynomial Algorithm

- We can reduce the message size to polynomial with a simple algorithm
- The number of processors increases to $n > 4f$
- The number of rounds increases to $2(f + 1)$
- Uses $f + 1$ phases, each taking two rounds

Phase King Algorithm

Code for each processor P_i :

pref := my input

Round 1 of phase k , $1 \leq k \leq f+1$:

send pref to all

receive prefs of others

let majority be value that occurs $> n/2$ times (0 if none)

let multiplicity be number of times majority occurs

Round 2 of phase k :

if $i = k$ then send majority to all // P_k is the king for phase k

receive tie-breaker from p_k (0 if none)

if multiplicity $> n/2 + f$

then pref := majority

else pref := tie-breaker

if $k = f + 1$ then decide pref

Unanimous Phase Lemma

- If all non-faulty processors prefer v at start of phase k , then all do at end of phase k .
- **Proof:**
- Each nonfaulty proc. receives at least $n - f$ preferences for v in first round of phase k
- Since $n > 4f$, it follows that $n - f > n/2 + f$
- So each nonfaulty proc. still prefers v .

Phase King Validity

- Unanimous phase lemma implies validity:
- Suppose all procs have input v .
- Then at start of phase 1, all non-faulty procs prefer v .
- So at end of phase 1, all non-faulty procs prefer v .
- So at start of phase 2, all non-faulty procs prefer v .
- So at end of phase 2, all non-faulty procs prefer v .
- ...
- At end of phase $f + 1$, all non-faulty procs prefer v and decide v .

Nonfaulty King Lemma

- If king of phase k is nonfaulty, then all nonfaulty procs have same preference at end of phase k .
- **Proof:** Let P_i and P_j be nonfaulty.
- *Case 1:* P_i and P_j both use p_k 's tie-breaker. Since P_k is nonfaulty,
- Thus, both P_i and P_j have the same preference.

Nonfaulty King Lemma

- *Case 2:* P_i uses its majority value v and P_j uses king's tie-breaker.
- Then P_i receives more than $n/2 + f$ preferences for v
- So P_k receives more than $n/2$ preferences for v
- So P_k 's tie-breaker is v .

Nonfaulty King Lemma

- *Case 3:* P_i and P_j both use their own majority values.
- Suppose P_i 's majority value is v
- Then P_i receives more than $n/2 + f$ preferences for v
- So P_j receives more than $n/2$ preferences for v
- So P_j 's majority value is also v .

Phase King Agreement

- Use previous two lemmas to prove agreement:
- Since there are $f + 1$ phases, at least one has a nonfaulty king.
- Nonfaulty King Lemma implies at the end of that phase, all nonfaulty processors have same preference
- Unanimous Phase Lemma implies that from that phase onward, all nonfaulty processors have same preference
- Thus, all nonfaulty decisions are same.

Summary of Phase King Algorithm

- Number of processors $n > 4f$
- Number of rounds = $2(f + 1)$ rounds
- Number of messages = $O(n^2f)$.

One Table to Fill

Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous	Easy, All-to-All communication	$f+1$ rounds	• Possible if $n > 3f$, $O(n^f)$ messages. • Possible if $n > 4f$, $O(n^2f)$ messages, $f+1$ rounds
Asynchronous	Possible, to do later		

Asynchronous Systems and Failures

- Let us consider a faulty asynchronous system and the consensus problem.
- The nature of fault really does not matter.
- Even under a single crash fault, it is known that in asynchronous systems, it is **impossible** to arrive at consensus.
- The result is due to Fischer, Lynch, and Paterson.
- Called the FLP result.

One Table to Fill

Fault/ Comm. Model	Fault-Free	Faulty	
		Fail-Stop	Byzantine
Synchronous	Easy, All-to-All communication	$f+1$ rounds	• Possible if $n > 3f$, $O(n^f)$ messages. • Possible if $n > 4f$, $O(n^2f)$ messages, $f+1$ rounds
Asynchronous	Possible, to do later	Not Possible	Not Possible

Distributed Systems
Monsoon 2025
Lecture 14

International Institute of Information Technology
Hyderabad, India

Some Logistics

- HW 3 to be posted today. Due Monday, October 6, 2025.
- Course Project:
 - Project list to be posted by Monday morning.
 - Make your team of 2 or 3.
 - Will release a form to put in your teammates, project preferences, etc.
 - Project assignment follows soon after.

The FLP result

- The result is a bit **devastating** for practical systems.
- Most practical systems are **asynchronous**.
- The result says that many tasks including leader election are not possible in such systems.
- But, we know of several practical systems that do use consensus style mechanisms.
 - Various database protocols including UPI transactions
- So, there seems to be a **middle-ground**!

PAXOS

- Paxos has three roles for nodes
 - Proposers : The job of a proposer is to propose values that will be considered by the consensus protocol.
 - Acceptors: The job of an acceptor is to temporarily accept values and help the protocol terminates.
 - Learners: A learner tries to learn the value that has been chosen as the consensus value

The Practical World – PAXOS

- A set of processes that can propose values
- Processes can crash and recover
- Processes have access to stable storage
- **Asynchronous** communication via messages
- Messages can be lost and duplicated, but not corrupted

The Practical World – PAXOS

- Two conditions to be satisfied:
- SAFETY: (also equivalent to Validity)
 - Only a value that has been proposed can be chosen
 - Only a single value is chosen
 - A process never learns that a value has been chosen unless it has been
- LIVENESS (also equivalent to Agreement+ Termination)
 - Some proposed value is eventually chosen
 - If a value is chosen, a process eventually learns it

How to Choose a value?

- With a single acceptor.
- The acceptor looks at all proposals and chooses a value.
- Works fine until the acceptor fails.
 - No other process has knowledge of the choice
- More like a centralized solution. Not good enough for a distributed setting.

How to Choose a value?

- Use multiple acceptors.
- Choose only when a “large enough” or a “majority” set of acceptors accept
- Using a majority set guarantees that at most one value is chosen.
- Two majority sets will always have at least one overlap!

How to Accept a value?

- Use multiple acceptors.
- Suppose there is a single proposer.
- Suppose only one value is proposed by the single proposer.
- That value should be chosen!
- First Rule of Paxos – R1:
 - **First-Accept Rule** : An acceptor must accept the first proposal that it receives.
 - This serves the situation when the acceptor just recovers from a fault also.

How to Accept a value?

- First Rule of Paxos – R1:
 - **First-Accept Rule** : An acceptor must accept the first proposal that it receives.
 - This serves the situation when the acceptor just recovers from a fault also.
 - Does not know if this will be the last proposal message or not.
 - It very well may be the last proposal that is received.

How to Accept a value?

- First Rule of Paxos – R1:
 - **First-Accept Rule** : An acceptor must accept the first proposal that it receives.
 - This serves the situation when the acceptor just recovers from a fault also.
 - Does not know if this will be the last proposal message or not.
 - It very well may be the last proposal that is received.
- But what if we have multiple proposers, each proposing a different value?

How to Accept a Value

- Acceptors must (be able to) accept more than one proposal
- To keep track of different proposals, assign a natural number to each proposal
 - A proposal is then a tuple (pnumber, value)
 - Different proposals have different pnumbers.
 - A **proposal is chosen** when it has been accepted by a majority of acceptors
 - A **value is chosen** when a single proposal with that value has been chosen

Paxos

- We need to guarantee that all chosen proposals result in choosing the same value
- We introduce a second Rule, called the Consensus Rule.
- **Consensus Rule R2** -- If a proposal with (pnumber, v) is chosen, then **every** higher-numbered proposal that is chosen has value v.

which can be satisfied by:

- **R2a**. If a proposal with (pnumber, v) is chosen, then every higher-numbered proposal accepted by **any** acceptor also has value v.

Paxos

- R2. If a proposal (pnumber, v) is chosen, then **every** higher-numbered proposal that is chosen has value v.

which can be satisfied by:

- R2a. If a proposal (pnumber, v) is chosen, then every higher-numbered proposal accepted by any acceptor also has value v

Which can be tightened as

- R2b: If a proposal (pnumber, v) is chosen, then every higher-numbered proposal **issued** by any proposer has value v

Paxos

- R2b: If a proposal (pnumber, v) is chosen, then every higher-numbered proposal **issued by any proposer** has value v.
- Suppose a proposer p wants to issue a proposal numbered n. What value should p propose?
- If (n', v) with $n' < n$ is chosen, then in every majority set S of acceptors at least one acceptor has accepted (n', v) .
- So, if there is a majority set S where no acceptor has accepted (or will accept) a proposal with number less than n, then p can propose any value

Paxos

- R2b: If a proposal with value v is chosen, then every higher-numbered proposal **issued by any proposer** has value v .
- What if for all S some acceptor ends up accepting a pair (n', v) with $n' < n$?
- Claim: p should propose the value of the highest numbered proposal among all accepted proposals numbered less than n
- Proof: By induction on the number of proposals issued after a proposal is chosen

Paxos

- **R2b**: If a proposal (pnumber, v) is chosen, then every higher-numbered proposal **issued by any proposer** has value v.
- Achieved by enforcing the following invariant R2c:
- **R2c**: For any v and n, if a proposal with value v and number n is issued, then there is a set S consisting of a majority of acceptors such that either:
 - no acceptor in S has accepted any proposal numbered less than n, or
 - v is the value of the highest-numbered proposal among all proposals numbered less than n accepted by the acceptors in S

Paxos

- To maintain R2c, a proposer that wishes to propose a proposal numbered n must **learn** the highest-numbered proposal with number less than n , if any, that has been or will be accepted by each acceptor in some majority of acceptors.
- Avoid predicting the future by **extracting a promise** from a majority of acceptors not to subsequently accept any proposals numbered less than n .

The Proposer's Protocol

- A proposer chooses a new proposal number n and sends a request to each member of some (majority) set of acceptors, asking it to respond with:
 - A promise never again to accept a proposal numbered less than n , and
 - The accepted proposal with highest number less than n if any. ..
- This is called a **prepare** request with number n .

Paxos – The Protocol in Pseudocode

Algorithm Paxos: Phase 1

```
1: procedure PROPOSE(v)
2:   ▷ Called by the proposer
3:   cnt ++
4:   val ← v
5:   Send ⟨prepare(cnt)⟩ to a majority of acceptors
6: end procedure
7:
8: procedure RECEIVEPREPARE(n)
9:   ▷ Called by the acceptor
10:  if n > maxPrep then
11:    maxPrep ← n
12:    return maxAccept
13:  end if
14: end procedure
```

Choosing a Value

- A proposer chooses a new n and sends to a majority of acceptors
- If an acceptor a receives $\langle \text{Prepare}, n', v' \rangle$, where $n' > n$ of any $\langle \text{Prepare}, n, v \rangle$ to which it has responded, then it responds to $\langle \text{Prepare}, n', v' \rangle$, with
 - a promise not to accept any more proposals numbered less than n'
 - the highest numbered proposal (if any) that it has accepted

On Receiving a Proposal

- Calls the ReceivePrepare function.
- Compare the number of the proposal with the internal variable maxPrep, which is the highest proposal number that it has received.
- Check if $n > \text{maxPrep}$ and consider newer proposals.
 - In this case, it sets maxPrep to n and returns a message with the value of the internal variable maxAccept

Phase 2 of the Protocol

- If the proposer receives a response from a majority of acceptors, then it can issue an accept message with number n and value v , where v is
 - the value of the highest-numbered proposal among the responses, or
 - any value selected by the proposer if responders returned no proposals
- A proposer issues an accept message by sending, to a majority set of acceptors, a request that the proposal be accepted.

Pseudocode

Algorithm Paxos: Phase 2

```
1: procedure RECEIVEMAXACCEPTS(maxAccepts[])
2:   ▷ Called by the proposer
3:   ▷ received maxAccept messages from a majority of acceptors
4:    $(n, v) \leftarrow \max(\text{maxAccepts})$ 
5:   if  $(n, v) = \phi$  then
6:      $(n, v) \leftarrow \text{cnt}, \text{val}$ 
7:   else
8:      $(\text{cnt}, \text{val}) \leftarrow (n, v)$ 
9:   end if
10:  send  $\langle \text{accept}(n, v) \rangle$  to all the acceptors in the quorum
11: end procedure
```

The Acceptor's Protocol

- An acceptor receives an accept request from a proposer.
- It can respond to an accept request with number n , accepting the proposal, iff it **has not responded** to a prepare request having number greater than n .
 - Lower numbered requests do not get a reply!

Towards Consensus

- If the proposer receives a response to $\langle \text{Prepare}, n, v \rangle$ from a **majority** of acceptors, then it sends $\langle \text{Accept}, n, v \rangle$ to each acceptor,
 - where v is either the value of the highest numbered proposal among the responses
 - any value if the responses reported no proposals
- If an acceptor receives $\langle \text{Accept}, n, v \rangle$, it accepts the proposal unless it has in the meantime responded to $\langle \text{prepare}, n' \rangle$, where $n' > n$

Pseudocode Phase 2 (Contd...)

```
13: procedure RECEIVEACCEPT( $n, v$ )
14:     ▷ Called by the acceptor
15:     if  $n \geq \text{maxPrep}$  then
16:          $\text{maxAccept} \leftarrow (n, v)$ 
17:         accept the proposal  $(n, v)$ 
18:         send a  $\langle \text{response} \rangle$  to the proposer
19:     end if
20: end procedure
21:
22: procedure RECEIVEDALLRESPONSES
23:     ▷ Called by the proposer
24:     choose  $val$ 
25: end procedure
```

Liveness

- This scheme does not guarantee progress.
- Consider two proposers P1 and P2 who have to keep making proposals with increasing value, none of which are accepted.
- Key to progress is to have only proposer at a time.

Liveness

- Key to progress is to have only proposer at a time.
- To achieve that, in normal operation, elect a single server to be a **leader**.
- The leader acts as the distinguished proposer in all instances of the algorithm.

How about the FLP Result?

- What did Paxos do that FLP result of impossibility is bypassed?
- Paxos is always safe, but
- Paxos, for progress, **needs a leader**.
- But, leader election is difficult in the asynchronous setting.
- If there are **enough periods of synchrony**, then Paxos is both safe and live.

Other Protocols

- Paxos was deemed too difficult to understand completely.
- Some more simpler protocols proposed over the years
 - RAFT
 - PBFT
 - ...
- RAFT does not tolerate Byzantine faults.